

Доклад по BPMN (Business Process Model and Notation)

1. Что такое BPMN?

BPMN (Business Process Model and Notation) — это стандартный графический язык моделирования бизнес-процессов, разработанный организацией OMG (Object Management Group). Первая версия вышла в 2004 году, а в 2011 году появилась актуальная версия BPMN 2.0, которая используется по сей день.

BPMN представляет собой **систему условных обозначений** для описания бизнес-процессов — деятельности, происходящей внутри компании. Открытие счетов, приём и увольнение сотрудников, согласование документов — все эти действия можно рассматривать как бизнес-процесс либо его часть.

2. Зачем нужна BPMN?

Главная ценность стандарта — **единый язык между бизнесом и ИТ**. Аналитик рисует схему процесса, разработчик смотрит на ту же схему и понимает, как автоматизировать процесс. Никаких «я имел в виду другое» и потерянных деталей при передаче задачи.

Ключевые преимущества:

- **Открытый стандарт** — поддерживается большинством BPM-систем
- **Наглядность** — диаграммы понятны как бизнес-пользователям, так и разработчикам
- **Исполнимость** — диаграмму, созданную по правилам, BPM-система может прочитать и запустить как автоматизированный процесс
- **Гибкость** — построить процесс может не только ИТ-специалист, но и рядовой пользователь

3. Основные элементы BPMN

Элементы BPMN делятся на несколько групп:

3.1. Объекты потока (Flow Objects)

Элемент	Обозначение	Описание
События	Круги	Фиксируют точки, где что-то происходит. Бывают трёх типов: стартовое (тонкий круг), промежуточное (двойной круг), конечное (жирный круг)
Задачи/Действия	Прямоугольники со скруглёнными углами	Конкретная работа, которую нужно выполнить
Шлюзы	Ромбы	Логические операторы для контроля расхождений и схождений потока: «И», «ИЛИ», «исключающее ИЛИ»

3.2. Соединяющие объекты (Connecting Objects)

- **Поток управления** — показывает последовательность выполнения
- **Поток сообщений** — показывает обмен сообщениями между участниками
- **Ассоциации** — связывают артефакты с объектами потока

3.3. Пули (Swim Lanes)

- **Пулы** — крупные контейнеры для участников процесса
- **Дорожки** — более детальное разделение внутри пула (например, по отделам)

3.4. Артефакты (Artifacts)

- **Объекты данных** — информация, которая создаётся или используется в процессе
- **Группы** — объединение нескольких элементов для повышения наглядности
- **Текстовые аннотации** — комментарии и пояснения

4. Сравнение с другими нотациями

Нотация	Особенности
IDEF0	Детально отражает функции и потоки, но тяжело читается без подготовки
EPC	Интуитивно понятна благодаря цветовому кодированию, но каждое действие требует своего события
BPMN 2.0	Баланс наглядности и точности — понятна бизнесу и достаточна для технической реализации

5. Пример использования

Типичный процесс в нотации BPMN может выглядеть так: стартовое событие («поступила заявка») → задача («обработать заявку») → шлюз («проверка условий») → либо задача («одобрить»), либо задача («отклонить») → конечное событие («процесс завершён»).

Конспект: Виды зависимостей в UML

Общее понятие

Зависимость (Dependency) — это отношение, показывающее, что изменение в спецификации одного элемента (независимого) может повлиять на поведение другого элемента (зависимого), который использует первый. Зависимость — это связь **по применению**.

В UML зависимость изображается **пунктирной линией со стрелкой**, направленной от зависимого элемента к независимому. Это **самый общий тип направленного отношения**.

UML определяет **4 базовых типа зависимостей**:

1. **Использования (Usage)** — элемент использует услуги другого
2. **Абстракции (Abstraction)** — связь между разными уровнями абстракции
3. **Авторизации (Permission)** — доступ к элементу
4. **Привязки (Binding)** — подстановка параметров в шаблон

Таблица видов зависимостей UML (без примеров кода)

Вид зависимости	Стереотип	Описание в предметной области
Использование	«use»	Зависимость самого общего вида: класс каким-либо образом использует другой класс.
Вызов	«call»	Операция зависимого класса вызывает операцию независимого класса.
Создание экземпляра	«instantiate»	Операции зависимого класса создают экземпляры независимого класса.
Параметр	«parameter»	Операция зависимого класса использует объект независимого класса в качестве параметра.
Отправка	«send»	Зависимый класс отправляет сигнал независимому классу.
Привязка	«bind»	Подстановка аргументов в шаблон (параметризованный класс).
Производный	«derive»	Зависимый элемент может быть вычислен на основе независимого.
Уточнение	«refine»	Зависимый класс уточняет (конкретизирует) независимый — классы на разных уровнях абстракции.
Друг	«friend»	Зависимый класс имеет доступ к закрытым составляющим независимого класса.
Трассировка	«trace»	Связь между элементами разных моделей (например, требования и реализация).

Примеры кода Python для каждого вида зависимости (по порядку)

Ниже приведены примеры, иллюстрирующие каждый из перечисленных видов зависимостей на языке Python.

1. Использование («use»)

Класс `ReportGenerator` использует данные, полученные из другого источника (например, из объекта `data`), но не хранит ссылку на него как поле.

```
class ReportGenerator:
    def generate(self, data):
        # использует данные из другого класса
        return f"Report: {data}"
```

2. Вызов («call»)

Метод `process` класса `OrderService` вызывает метод `process_payment` класса `PaymentGateway`.

```
class PaymentGateway:
    @staticmethod
    def process_payment(order):
        # обработка платежа
        return True

class OrderService:
    def process(self, order):
        result = PaymentGateway.process_payment(order) # вызов метода другого
        класса
        return result
```

3. Создание экземпляра («**instantiate**»)

Фабрика **OrderFactory** создаёт объекты класса **Order**.

```
class Order:
    def __init__(self, items):
        self.items = items

class OrderFactory:
    def create_order(self, items):
        return Order(items) # создание экземпляра другого класса
```

4. Параметр («**parameter**»)

Метод **calculate_cost** класса **ShippingService** принимает объект класса **Address** в качестве параметра.

```
class Address:
    def __init__(self, city, distance):
        self.city = city
        self.distance = distance

class ShippingService:
    def calculate_cost(self, address: Address) -> float:
        # Address используется как параметр
        return address.distance * 0.5
```

5. Отправка («**send**»)

Класс **Button** отправляет сигнал через шину событий.

```
class EventBus:
    @staticmethod
    def send(signal):
```

```
print(f"Signal sent: {signal}")

class Button:
    def click(self):
        EventBus.send("button_clicked") # отправка сигнала
```

6. Привязка («bind»)

Параметризованный класс `Repository` привязывается к конкретному типу `User` в `UserRepository`.

```
from typing import TypeVar, Generic

T = TypeVar('T')

class Repository(Generic[T]):
    def get(self, id: int) -> T:
        # возвращает объект типа T
        pass

# Привязка: UserRepository использует шаблон Repository с типом User
class UserRepository(Repository['User']):
    def get(self, id: int) -> 'User':
        # реализация
        pass
```

7. Производный («derive»)

Свойство `area` класса `Rectangle` вычисляется из атрибутов `width` и `height`.

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    @property
    def area(self):
        return self.width * self.height # площадь вычисляется из ширины и высоты
```

8. Уточнение («refine»)

Абстрактный класс `PaymentProcessor` уточняется конкретной реализацией `StripePaymentProcessor`.

```
from abc import ABC, abstractmethod

class PaymentProcessor(ABC):
    @abstractmethod
    def process(self, amount):
```

```
pass

# Уточнение (конкретный уровень)
class StripePaymentProcessor(PaymentProcessor):
    def process(self, amount):
        # реализация через Stripe
        return f"Charged {amount} via Stripe"
```

9. Друг («friend»)

Класс `Inspector` получает доступ к приватному атрибуту класса `SecretData` (в Python соглашение о приватности с `_` или `__` не является строгим).

```
class SecretData:
    def __init__(self):
        self._secret = "classified"

class Inspector:
    def reveal(self, obj):
        return obj._secret # доступ к "приватному" атрибуту
```

10. Трассировка («trace»)

Связь между требованием (например, в документации) и его реализацией в коде.

```
# Требование: система должна аутентифицировать пользователя
# Реализация:
class AuthService:
    def login(self, username, password):
        # проверка учетных данных
        return True
```

Дополнительные отношения в UML (независимо от стереотипов зависимости)

Хотя эти отношения часто рассматриваются отдельно, они также могут быть выражены через зависимости или являются более сильными формами связей.

Ассоциация

Постоянная структурная связь между объектами классов (объект одного класса хранит ссылку на объект другого).

```
class Department:
    pass
```

```
class Employee:
    def __init__(self):
        self.department = None # ссылка на объект Department
```

Агрегация

Отношение «часть-целое» со слабой связью: части могут существовать отдельно от целого.

```
class Employee:
    pass

class Team:
    def __init__(self):
        self.members = [] # список Employee (сотрудники могут существовать без команды)
```

Композиция

Отношение «часть-целое» с сильной связью: части не существуют без целого.

```
class Room:
    pass

class House:
    def __init__(self):
        self.rooms = [Room() for _ in range(5)] # комнаты создаются вместе с домом
```

Обобщение (наследование)

Отношение «is-a» — один класс является специализацией другого.

```
class Animal:
    pass

class Dog(Animal): # Dog наследует Animal
    pass
```

Реализация

Класс реализует интерфейс (абстрактный класс).

```
from abc import ABC, abstractmethod
```

```
class Drawable(ABC):
    @abstractmethod
    def draw(self):
        pass

class Circle(Drawable): # реализация интерфейса
    def draw(self):
        print("Drawing circle")
```